

SSA Permission Matrix

Authoritative reference. The database side lives in `supabase/migrations/0002_rls_policies.sql` and the migrations that amend it; the app repeats the same rules in the UI so people are not offered controls that will fail.

Re-derived from the code and the live policies on 23 Sep 2026. The tables below were rebuilt by reading `src/lib/rolePermissions.js`, `src/lib/phaseRoles.ts`, `src/lib/shareRoles.ts`, `src/components/BoatConfigTab.tsx` and the RLS policies themselves, then checking the upload rules against the live database role by role. The previous version predated `tl3` and `owner` and was wrong in several places.

Role definitions

Role	Scope	Created by	Time-windowed?
<code>admin</code>	global	manual SQL on <code>users.global_role</code>	No
<code>team_manager</code>	per team	admin / team manager assigns membership	No
<code>coach</code>	per (team, boat)	admin / team manager assigns membership	No
<code>tl3</code>	per (team, boat)	admin / team manager assigns membership	No
<code>tl2</code>	per (team, boat)	admin / team manager assigns membership	No
<code>tl1</code>	per (team, boat)	admin / team manager assigns membership	No
<code>owner</code>	per (team, boat)	admin / team manager assigns membership	No
<code>consultant</code>	per (team, boat)	admin / team manager assigns membership	Yes — <code>valid_from</code> / <code>valid_to</code>
<code>guest</code>	per (team, boat)	admin / team manager assigns membership	Optional

`admin` is a global flag on the user, not a membership. Everyone else holds one or more memberships and switches between them in the app.

The sailing ladder is `tl1` → `tl2` → `tl3`, `tl3` being the most senior. It is not obvious from the names, and reading it backwards is how `tl3` ended up unable to upload while `tl1` could (fixed in 0079/0080).

What decides what you can SEE

Reads are **not** gated by role. `has_boat_access(team_id, boat_id)` asks only whether you hold a live membership covering that boat:

- a membership with `boat_id` set → that boat only
- a membership with `boat_id` NULL → every boat in the team
- outside `valid_from` ... `valid_to` → nothing

So "can a `tl1` see the other boat's day?" is a question about how their membership was created, not about being a `tl1`. Role decides what you can *do* and which *tabs* you get.

Tabs and features the UI offers

From `src/lib/rolePermissions.js` unless noted. These decide what is OFFERED; RLS is the boundary.

	<code>admin</code>	<code>team_manager</code>	<code>coach</code>	<code>tl3</code>	<code>tl2</code>	<code>tl1</code>	<code>owner</code>	<code>consultant</code>	<code>guest</code>
Timeline / day view	✓	✓	✓	✓	✓	✓	✓	🕒	latest day only
Analytics tab (GPS map)	✓	✓	✓	✓	✓	✓	✓	🕒	✓
Analytics — charts, polars, data	✓	✓	✓	✓	✓	✗	✗	🕒	✗
SailScan tab	✓	✓	✓	✓	✓	✗	✗	🕒	✗

	admin	team_manager	coach	tl3	tl2	tl1	owner	consultant	guest
SquashShots tab	✓	✓	✓	✓	✓	✓	✓	🕒	✗
Tools tab (Squash + SailScan)	✓	✓	✓	✓	✓	✗	✗	🕒	✗
Photos — SailScan-tagged photos	✓	✓	✓	✓	✓	✗	✗	🕒	✗
Boat Config tab	✓	✓	✓	✓	✗	✗	✗	🕒 partial ¹	✗
AI (debrief summary, AI search)	✓	✓	✓	✓	✓	✗	✗	✗ ²	✗

¹ A consultant sees Boat Config but only the **Sail inventory** and **Sail data** sub-tabs; Rig settings, Targets and Log profile are hidden (`canSeeTuning` , `BoatConfigTab.tsx`). A sailmaker gets what they came for and not the tuning.

² Deliberate: AI is metered per team, and a consultant is not the team.

What each role can DO

Enforced by RLS. 🕒 = allowed only inside the consultant's date window.

Action	admin	team_manager	coach	tl3	tl2	tl1	owner	consultant	guest
Upload log + event file (<code>sessions</code>)	✓	✗	✓	✓	✗	✗	✗	🕒	✗
Upload video	✓	✗	✓	✓	✗	✗	✗	🕒	✗
Upload photos	✓	✗	✓	✓	✗	✗	✗	🕒	✗
Upload rig / mast settings	✓	✗	✓	✓	✗	✗	✗	🕒	✗
Edit a row you created	✓	✓ ³	✓	✓	✓	✓	✓	🕒	✓
Edit anyone's row	✓	✗	✓	✗	✗	✗	✗	✗	✗
Delete uploads	✓	✗	✓	✗	✗	✗	✗	✗	✗
Edit Boat Config (rig, sails, targets)	✓	✓	✓	✓	✗	✗	✗	✗	✗
Build phases from a day	✓	✓	✓	✓	✓	✗	✗	✗	✗
Upload a phase set	✓	✓	✓	✗	✗	✗	✗	✗	✗
Share a clip outside the team	✓	✓	✓	✓	✓	✗	✓	✗	✗
Write the campaign plan / day	✓	✓	✓	✓	✗ ⁴	✗ ⁴	✗	✗	✗
Counted for debrief-recording consent	—	✗	✓	✓	✓	✓	✗	✗	✗

³ `own_or_coach()` names no roles: whoever created a row may edit it, and a coach may edit anyone's. A `team_manager` therefore edits only what they created.

⁴ Campaign **spine**, **debrief backlog**, **debrief notes** and **manoeuvre events** (0015, 0016, 0017, 0019) still say `ARRAY['coach','tl1','tl2']` — so tl1/tl2 can write those while **tl3 cannot**, which is the same omission 0079 fixed for uploads. Known, not yet fixed; it needs its own reviewed change.

Why uploads are tl3 and up

Putting a day into the archive is a senior job — a bad import overwrites a day for the whole team (re-importing a log replaces what is stored). tl3 exists precisely so a squad can hand that to its senior sailors: on the dinghy side the coach does not hold every sailor's tracker, so each sailor uploads their own track to their own boat.

`team_manager` is excluded on purpose, since 0007. One who also sails holds a second membership — coach or tl3 — for that.

`consultant` keeps uploads, bounded by `valid_from/valid_to` . A sailmaker who cannot upload the scan they came to take is no use, and the date window is the control rather than the role.

Resource x role matrix — administration

The tables in this section cover accounts, teams and memberships and predate `tl3` and `owner` ; where they disagree with the two tables above, the ones above are correct.

Legend:  allowed ·  forbidden ·  allowed only inside `valid_from ... valid_to` .

User profiles & approvals

Action	admin	coach	tl2	tl1	consultant
See own profile					
See teammates' profile (name, email)					
Edit own profile					
Approve pending users					
Suspend / disable users					

Teams & boats

Action	admin	coach	tl2	tl1	consultant
Create team					
Rename / delete team					
View team metadata					
Create boat in team					
Edit / delete boat					
View boat metadata					

Memberships (who's in a team)

Action	admin	coach	tl2	tl1	consultant
See own memberships					
See team's memberships					
Add / remove memberships					
Change role of a membership					
Set consultant time window					

Coaches can request membership changes via admin out-of-band; the system does not let them mutate memberships directly.

Sessions, photos, videos, mast_settings, tag_lists

Superseded — see **What each role can DO** above, which is the current and verified version. Tables ship in `0003_data_schema.sql` ; uploads were narrowed to tl3-and-up in `0080_uploads_tl3_and_up.sql` .

Reads follow `has_boat_access` (membership + boat + date window), not role. Consultant writes are bounded by `valid_from / valid_to` : when the window closes, RLS denies reads and writes alike, so they can no longer see or edit anything they contributed — the data itself stays in the team archive.

Analyses (SailScan, SquashShots, AI commentary)

These are UI-level gates (the DB doesn't know about features). Enforced in client components via the active membership's role.

Action	admin	team_manager	coach	tl2	tl1	consultant
Run SailScan on a photo	✓	✗	✓	✓	✓	🕒
Run SquashShots	✓	✗	✓	✓	✓	🕒
Run AI commentary / video AI	✓	✗	✓	✓	✗	✗
Access Data Analysis tab	✓	✓	✓	✓	✓	✗
View existing analyses	✓	✓	✓	✓	✓	🕒
Edit / approve a SailScan result	✓	✗	✓	✓	✓	✗
Calibrate yacht stripe colours	✓	✗	✓	✓	✗	✗

Rationale: tl1 + consultant can take pictures and run SailScan / SquashShots (cheap inference). AI commentary stays gated to coach + tl2 (more expensive, requires interpretation skill). Consultants are deliberately blocked from the data-analysis tab — they're external advisors and should view through the lens the team curates for them, not browse the raw archive.

Quotas & storage

Action	admin	coach	tl2	tl1	consultant
See own quota	✓	✓	✓	✓	✓
See others' quota	✓	✗	✗	✗	✗
Override a user's <code>bytes_limit</code>	✓	✗	✗	✗	✗
Reset 80 / 100 % warning flags	✓	✗	✗	✗	✗

Default `bytes_limit` per role: see `spec.md § Quotas`.

Audit / events

Action	admin	coach	tl2	tl1	consultant
Append events for self	✓	✓	✓	✓	✓
View own events	✓	✓	✓	✓	✓
View team's events	✓	✗	✗	✗	✗
Edit / delete events	✓	✗	✗	✗	✗

Notes on the consultant time-window

`memberships.valid_from` / `valid_to` define the *active* window. Outside that window, every helper function (`is_team_member`, `has_team_role`, `has_boat_access`) returns false, which means RLS denies SELECT on every team-scoped resource. The app should reflect this by hiding teams whose membership is expired in the team-switcher.

When data tables (sessions, photos, videos) land in L3, we'll add a *secondary* time filter: consultants only see rows whose timestamp `utc` falls inside the `valid_from` / `valid_to` window. So even within an active membership, consultants can't browse data outside their contracted period.

What "active user" means

An "active" user is one with `users.status = 'active'`. The helper `is_active_user()` is used by event-insert policies to make sure pending / disabled users can't append to the audit log. Most queries don't need this — RLS denies them anyway because they have no memberships and `is_admin()` returns false.